
Dr. Pooja's Physio - Operations Admin Test Plan

The operations desk: what it opens, what it changes, and what it is refused

Role

Operations Admin

Document version

1.0

Audience

A tester checking this role end to end

Environment

Throwaway Supabase project · Razorpay test mode · npm run dev

Read first

Section 1 - who this is, and what has to exist before you start

Contents

1. Who this is, and what must exist first
2. Signing in, and what the screen says you are
3. Today
4. Sessions - manage, which is the whole of this desk's day
5. People - manage, with one control that is not theirs
6. Catalog - manage
7. What Operations must never reach
8. Suspension, and the scope itself
9. Sign-off

1. Who this is, and what must exist first

The desk. Operations runs the day: sessions, the people in them, and the catalogue they are booked against. They do not touch the books, they cannot read the activity log, and they cannot change how the product behaves.

What this desk opens, exactly. Four of the seven sections, all at the level that changes things:

Section	Operations
Today	manage
Sessions	manage
People	manage
Catalog	manage
Money	none
Logs	none
Settings	none

That table is the whole of this plan. Everything below either confirms a capability that grid grants, or confirms a refusal it implies - at the screen **and** at the route, because a session cookie can call a route around a hidden button.

The account.

Label	Email	Password
Operations	qa.admin.ops@example.test	generated when the Master Admin creates it

The password is generated and shown once when the account is created. Write it down then.

1.1 Before you start

Must exist	Set up by
This Operations account, created from Settings → User Access	Master Admin
A catalogue: at least two treatment categories and one programme	Master Admin
Two approved therapists, one with a roster	Master Admin + Therapist
At least one patient with a paid, unassigned session	Patient
At least one patient with a completed session and a refund on it	Master Admin
A partner hospital with a revenue share and one referral	Master Admin + Hospital
One queued recommendation	Therapist

Two of those matter particularly. A refunded session is what proves this desk cannot read the money on it, and the partner hospital is what proves a section they *can* open still hides the controls that are not theirs.

1.2 How to record a result

Mark every step **Pass**, **Fail**, **Blocked** or **N/A**. Raise a failure with: the step id, what you did, what you expected, what happened, a screenshot, any console error in full, how many times out of how many it reproduced, and **which account you were signed in as**.

Severity. P0 - money, clinical data or access control is wrong: stop and report. **P1** - a main journey is broken with no workaround. **P2** - wrong with a workaround. **P3** - cosmetic.

In this document, a refusal that does not happen is almost always a P0. This desk exists to be narrower than the Master Admin's, so a control that works when it should not is the failure this plan is for.

1.3 Calling a route without a terminal

Most refusals here are checked twice - once by the screen hiding a control, once by the route refusing the call - and **only the second is the rule**. Sign in as Operations, press **F12**, open **Console**, paste and press Enter:

```
const r = await fetch("/api/some/route", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ some: "value" }),
});
({ status: r.status, body: await r.text() });
```

The request is same-origin, so the browser attaches this admin's cookie itself. Use a **second browser profile** for a different account, not a second tab - tabs share cookies. Never paste the URL in the address bar: that sends a GET, these are POSTs, and you will get 405.

2. Signing in, and what the screen says you are

0P-01 - The dashboard names itself · P2

Do this. Sign in as the Operations admin.

Expect

- The sidebar brand reads **Operations**, above "Admin Panel".
- The **eyebrow over the section heading** reads **Operations** too.
- **Both**, not one. The sidebar collapses to icons and is a closed drawer on a phone; the header is on every screen at every width. **A dashboard that just says "Admin Panel" is a P2** - four desks that differ only in which sidebar entries are missing make an admin infer which one they are on from an absence.

0P-02 - The "Your access" card · P2

Expect a card naming which sections this desk covers.

A shorter sidebar with no explanation reads as a fault. This card is the sentence saying the missing sections were hidden on purpose. **P2 if it is absent.**

It carries **no scope name of its own** - the brand and the header both say **Operations** on that same screen already.

0P-03 - The sidebar has four entries, and three are gone · P0

Expect in the sidebar: **Today, Sessions, People, Catalog.**

Expect absent: Money, Logs, Settings.

Any of those three rendering is a P1 at the screen level - and the routes below are what decide whether it is a P0.

3. Today

0P-04 - Operations leads with unassigned sessions · P1

Expect

- The headline figure is **unassigned sessions** - this desk's own first question. Finance leads with what is owed, Clinical with the recommendation queue, and that difference is deliberate.
- **No money figure is anywhere on this screen.** Not unlinked, not greyed - **the page does not compute it.** A rupee figure on an Operations Today is a **P1**.

0P-05 - "Needs you" agrees with the list under it · P2

Do this. Read the "needs a person" figure, then count the queue rows beneath it.

Expect

- They **match**. A figure that sums every queue while the list beside it is already filtered to what you can open is the commonest defect in this product's history.
- It counts only the queues this desk can **work**. A section readable but not manageable holds no work for them - a figure nothing they could do would ever bring down is worse than no figure.

0P-06 - Every quick action lands somewhere you can open · P1

Do this. Tap each quick action and come back.

Expect

- **Every one lands on a screen Operations can open.** An action for an unreachable section is **dropped**, never rendered.
- **None lands back on Today.** That is what a link into a section you cannot open does - it falls back to the first section you *can* open, silently. **A quick action that reloads Today is a P1**, and it is the exact failure this rule exists to catch.

0P-07 - The queue order is emphasis, never permission · P2

Expect. This desk's own domains sit at the top, and **nothing is removed**. What a scope may work is the routes' decision; a UI that hid a reachable queue would be a second permission model to disagree with the first. **A queue you can open that is missing from this list is a P2.**

0P-08 - Activity is their own desk's · P2

Do this. Open **Today** → **Activity**.

Expect

- Entries whose action belongs to a section **this desk can work** and whose **actor sits at this desk**.
- It will look **sparse** - in a small clinic the Master Admin does most of the work. That is why **the screen says the list is their desk's** rather than showing an empty panel that reads as "nothing happened". **P2 if it just looks empty.**
- **This is not the Logs section.** It is a feed of their own work, and it is the only exception to "ordering is emphasis, never permission" - a queue is work waiting on somebody, so hiding one hides their job; a log is a record of what other people did.

0P-09 - Risk, scoped by rule · P2

Do this. Open **Today** → **Risk**.

Expect

- The rules whose **domain** is a section this desk can act on - a contact leak and an early completion are sessions questions, so they are here. A cash variance is a money question and is **not**.
- **The flagged messages panel and the contact reveal log are absent.** Those quote what a colleague wrote and name every patient contact they opened, and stay Master-Admin-only. **Either rendering here is a P0.**
- The thresholds themselves are not editable here.
- **No action buttons on the tab**, same as every desk. A flag is never an accusation and never carries a penalty.

4. Sessions - manage, which is the whole of this desk's day

0P-10 - Assign, reassign, reschedule · P1

Do	Expect
Open the unassigned queue from Today	Exactly the rows it counted, with the other filters cleared first
Assign a session	Succeeds. Sessions is manage
Read the control on a never-assigned session	Tap to assign , not "Reschedule / Reassign"
Reschedule a confirmed session	Succeeds , with the same calendar and hour chips a patient sees
Try a time like 6:52	Cannot be produced at all
Book a new session inside the 12-hour window	Refused, until the override box is ticked
Book into the past with the box ticked	Refused

0P-11 - Complete and reopen · P1

Do	Expect
Complete a paid session on a therapist's behalf	Succeeds. An admin is subject to neither of the therapist's two gates
Reopen a completed session	Succeeds , warning first that both sides' ratings are destroyed

Do	Expect
Check the completion timestamp after reopening	Cleared , not left behind

0P-12 - Cancelling, and what they cannot see about the refund · P0

Do this. Cancel a paid session outside the 24-hour window.

Expect

- The cancellation **succeeds** - it is a Sessions capability.
- **Look hard at what the screen now tells you about the money.** The refund amount is a **money** reading; confirm what this desk sees against the Master Admin's view of the same row in another browser.
- Check the same on **All Sessions**: the **amount and the refund columns do not render** for this desk.

0P-13 - The export follows the screen · P0

Do this. Filter **All Sessions**, then export as **CSV** and as **PDF**.

Expect

- Both are offered, and both describe the **same filtered rows**.
- **Neither file contains the amount or the refund.** They do not render on screen for this desk, and **a scope enforced in the markup but not in the file the markup produces is not enforced**. An export is the easiest place to forget it. **Money in this file is a P0.**

0P-14 - The roster · P1

Do	Expect
Open Sessions → Roster	A list of therapists , not a calendar grid
Edit a weekly schedule	Succeeds , in periods rather than hourly cells
Write a date exception	Succeeds - this is an admin capability, and Operations has Sessions at manage
Set somebody on leave	Succeeds, and the weekly schedule survives intact
Remove hours a session sits in	It names who is affected and says the session stays as booked . P0 if it cancels or moves one

0P-15 - Recommendations · P1

Do	Expect
Open Sessions → Recommendations	The queue, oldest first, aged in words
Approve one	Succeeds
Turn one down with ok	Refused - ten characters
Approve one with different numbers	Succeeds , writing a new version attributed to the clinician and entered by you

Do	Expect
Write one on a therapist's behalf	Succeeds , with a mandatory reason, and no price, session-count or discount field anywhere on the form
Withdraw a purchased plan	Refused - the patient has paid and the sessions exist

A price input on that form is a P0, whichever desk is looking at it.

0P-16 - Delivery, and the figures that are not money · P2

Do this. Open **Sessions → Delivery**.

Expect. No-show rate, cancellation rate, repeat-booking rate, sessions per therapist. These live under **Sessions, not Money** - a no-show rate is about how the clinic runs, not about its books, which is exactly why this desk can read them while Money is closed to it.

5. People - manage, with one control that is not theirs

0P-17 - The directories · P1

Do	Expect
Open People → Patients , open one	Sessions, documents, notes - all readable and editable
Look at the money controls on that patient	They do not render
Approve a pending signup	Succeeds
Suspend an account	Succeeds
Reset somebody's password	Succeeds , with the password shown once and never in the log
Open People → Therapists	The directory, with the <code>/team</code> visibility control
Change a therapist's revenue share	There is no such control. That is a money capability

0P-18 - Partners, and the control that must not render · P1

Do this. Open **People → Partners** and open Hospital A.

Expect

- The partner's record, their referrals, and the referral capacity note - all editable.
- **The revenue-share editor and the two share figures do not render.** Partners sits on People, which this desk opens, but changing a revenue share is a **money** route. **A section a desk can open does not mean every control on it is theirs. P1 if it renders**, because pressing it produces a 403 with nothing on screen to explain it.

0P-19 - Deleting an account is full scope only · P0

Do this. Find the delete control on an account with no history at all, and press it.

Expect. Refused. Every desk that manages People can already suspend, and this one is irreversible - so it is checked against **full scope directly**, not through People. **Operations deleting an account is a P0.**

Then confirm at the route:

```
const r = await fetch("/api/admin/delete-account", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ userId: "PASTE_AN_ID" }),
});
({ status: r.status, body: await r.text() });
```

Expect a refusal.

0P-20 - Impersonation is full scope only · P0

Do this. Look for the impersonation control on a patient's profile.

Expect. Absent. Then call it:

```
const r = await fetch("/api/admin/start-impersonation", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ targetUserId: "PASTE_AN_ID", reason: "Checking a reported bug" }),
});
({ status: r.status, body: await r.text() });
```

Expect 403. This is checked against **full scope directly** rather than through a section, precisely so a section scope cannot hand the most dangerous capability in the product to whoever can edit a phone number. **A 200 here is a P0 that stops the run.**

6. Catalog - manage

0P-21 - Building and editing what the clinic sells · P1

Do	Expect
Create a treatment category	Succeeds , and reaches the public pages immediately
Reorder the list and Save order	Succeeds , renumbering the whole list
Upload and position a cover	Succeeds
Tick featured	Succeeds
Create a programme and a home-visit package	Succeeds
Add a service area with a travel fee	Succeeds
Delete a category that has bookings	Refused , in a dialog, naming and counting what is in the way and offering to turn it off instead

0P-22 - Purchases: the screen opens, the money controls do not · P0

Do this. Open **Catalog → Purchases** and open a programme purchase.

Expect

- The purchase detail opens - Catalog is `manage`.
- **The money controls do not render:** refund, grant credits, reverse a credit, revive an entitlement. A refund is a **money** capability even though its button lives on a Catalog screen - **the section is chosen by the capability, not by where the button happens to sit.**
- Extending an expiry and reassigning the locked therapist are catalogue actions and **do** render.

Confirm at the routes:

```
const routes = [
  "/api/admin/refund-package",
  "/api/admin/grant-session-credits",
  "/api/admin/reverse-session-credit",
  "/api/admin/revive-entitlement",
];
console.table(await Promise.all(routes.map(async (route) => {
  const r = await fetch(route, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: "{}",
  });
  return { route, status: r.status, body: (await r.text()).slice(0, 90) };
})));
```

Expect 403 on all four. Any 200 is a P0.

7. What Operations must never reach

0P-23 - Money, at the screen · P1

Try	Expect
Find Money in the sidebar	Absent
Open the Money URL directly	Refused, and the screen says their scope is why , in one dismissible amber line
Read that line	It names a screen that exists . Telling somebody they lack access to a screen that was never there is worse than silence
Use global search for a purchase or payment code	They are told it does not exist. A result opening a screen they cannot open would be a dead link that looked like it worked

A silent landing somewhere else is the failure here. The fallback landing somewhere valid is correct - a stale bookmark must not produce a blank page - but on its own the tap just goes somewhere and looks like it worked. **P2 if nothing is said.**

0P-24 - Money, at the routes · P0

```
const routes = [
  "/api/admin/settle-therapist-payout",
  "/api/admin/refund-session-partial",
  "/api/admin/apply-goodwill-discount",
  "/api/admin/correct-cash-amount",
  "/api/admin/mark-cash-remitted",
  "/api/admin/mark-cash-refund-returned",
  "/api/admin/update-hospital-revenue-share",
  "/api/admin/update-therapist-revenue-share",
  "/api/admin/save-promo-code",
  "/api/admin/delete-promo-code",
  "/api/admin/expenses",
  "/api/admin/finance",
  "/api/admin/mark-paid-by-cash",
];
console.table(await Promise.all(routes.map(async (route) => {
  const r = await fetch(route, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: "{}",
  });
  return { route, status: r.status, body: (await r.text()).slice(0, 90) };
})));
```

Expect 403 on every one. Not 404, not 500, and not a 400 that reads as "give me better arguments" - that is a route that checked the body before it checked who was asking. **Any 200 is a P0 that stops the run.**

0P-25 - Logs, at the screen and at both routes · P0

Try	Expect
Find Logs in the sidebar	Absent
Open the Logs URL	Refused, with the scope line

```
const routes = ["/api/admin/activity-log", "/api/admin/clear-activity-log"];
console.table(await Promise.all(routes.map(async (route) => {
  const r = await fetch(route, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: "{}",
  });
  return { route, status: r.status, body: (await r.text()).slice(0, 90) };
})));
```

Expect 403 on both. Logs is Master Admin's alone - it is the record of who impersonated whom, who settled which payout and who cleared the log. **A limited desk reading it is a P0.**

0P-26 - Settings, at the screen and at the routes · P0

Try	Expect
Find Settings in the sidebar	Absent
Open Settings → User Access by URL	Refused

Try	Expect
Open Settings → System Health by URL	Refused

```
const routes = [
  "/api/admin/update-setting",
  "/api/admin/set-admin-scope",
  "/api/admin/set-admin-active",
  "/api/admin/create-account",
  "/api/admin/save-mission-principle",
  "/api/admin/intake-questions",
  "/api/admin/debug-reset",
];
console.table(await Promise.all(routes.map(async (route) => {
  const r = await fetch(route, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: "{}",
  });
  return { route, status: r.status, body: (await r.text()).slice(0, 90) };
})));
```

Expect 403 on every one.

`create-account` matters most. The Back office group is absent from their picker, but that is **presentation** - the full-only check inside the route is what actually stops a limited scope minting an admin. **Operations creating an admin is a P0**, and it is a privilege escalation: they could mint themselves a Master Admin.

`debug-reset` matters second. Full scope only. **A limited desk wiping the database is a P0.**

OP-27 - And the other roles' doors · P0

```
const routes = [
  "/api/therapist/reveal-contact",
  "/api/therapist/suggest-session",
  "/api/patient/condition-profile/export",
  "/api/hospital/withdraw-referral",
];
console.table(await Promise.all(routes.map(async (route) => {
  const r = await fetch(route, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: "{}",
  });
  return { route, status: r.status, body: (await r.text()).slice(0, 90) };
})));
```

Expect a refusal on every one. An admin is not a therapist, and an admin route is not the same thing as every route.

8. Suspension, and the scope itself

0P-28 - Suspending this admin ends the session · P0

Do this

1. **As the Master Admin**, suspend the Operations admin.
2. In the Operations admin's **still-open browser**, reload the dashboard.
3. In the same browser, assign a session from the console.
4. Un-suspend.

Expect

- Step 2: bounced.
- Step 3: **refused**. A suspended admin's own routes refuse them too - the session was ended, not merely redirected away from. **A suspended admin still assigning sessions or reading patient records is a P0**: flipping a column without ending the session leaves them acting as themselves indefinitely, because a session cookie reaches the database without passing the screen.
- Step 4 restores them to **Operations**, not to a default and not to full.

0P-29 - Nobody changes their own scope · P0

```
const r = await fetch("/api/admin/set-admin-scope", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ userId: "YOUR_OWN_ID", scope: "full" }),
});
({ status: r.status, body: await r.text() });
```

Expect a refusal. This landing is a P0 and the most serious finding this document can produce - an admin who can widen their own scope has every capability in the product.

9. Sign-off

	Should be
Steps run	29
Pass	
Fail	
Blocked	
N/A	
P0 raised	
P1 raised	

Confirm each of these before signing.

The dashboard says Operations , twice, and carries the access card	☐
Four sidebar entries; Money, Logs and Settings absent	☐
Today leads with unassigned sessions and computes no money figure	☐
Every quick action landed somewhere they can open; none fell back to Today	☐
Sessions fully worked: assigned, rescheduled, completed, reopened, rostered	☐
Recommendations approved, refused and written on a therapist's behalf, with no price field	☐
People worked, but no revenue share, no delete, no impersonation	☐
Catalog worked, but no refund and no credit controls on Purchases	☐
The session export carried no amount and no refund	☐
403 on all thirteen money routes	☐
403 on both log routes	☐
403 on all seven settings routes, <code>create-account</code> and <code>debug-reset</code> included	☐
Suspension ended the session; the scope came back as Operations	☐
They could not widen their own scope	☐

Signed **Date**

Build / commit tested

What a clean run of this document proves. That the Operations scope grants what the grid says it grants and refuses what it says it refuses, at the screen and at the route, on one machine, once. It does not prove the other two desks are correctly narrowed - each has a document of its own - and it does not prove the capabilities themselves are correct, which is the Master Admin's document.